mb-free___transport-rendezvous-32768

An AgentMPI run analysis

Abstract

This is the extreme corner of a ten-cell transport sweep and its single strongest result. Eight agent-free ranks ring-exchange a 32,768-word payload under `Mode.RENDEZVOUS` — 43,116 estimated tokens, which is 1.32× the entire 32,768-token context budget of the rank receiving it. The run completed: 34 events, 0.08 s, all eight messages delivered, zero admissions, a context high water of zero and 0% occupancy on every rank, with 344,928 tokens deferred. A payload larger than a receiver's whole context window crossed the ring without occupying any of it. The eager cell at the same size produced 18 events, no messages and `ERR_CONTEXT_OVERFLOW` on all eight ranks, and it is over both limits at once: 10.5× the 4,096-token unexpected-message credit that actually refused it, and 1.32× the context budget, so it could not have succeeded under any credit setting. That asymmetry is the whole of AgentMPI's context-safety claim in one pair of runs, and this is the half that works.

1 What this run is

property	value
run	`mb-free__transport-rendezvous-32768`
experiment	-
campaign label	-
declared world size	8
ranks appearing in the log	8
executors	`none` × 8
events	34
wall time	0.07 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	0.00×	total busy time over wall time
parallel efficiency	0.0%	achieved parallelism per rank
idle fraction	100.0%	rank-seconds paid for and unused
peak ranks busy	0	of 8 in the world
serial fraction of active time	0.0%	time with exactly one rank busy
load imbalance	0.00×	slowest rank's busy time over the mean
coordination share	0.0%	rank-seconds blocked in collectives, of those available
coordination in flight	0.0%	wall clock with any rank inside a collective
rank reattachments	0	fresh agent processes that took over a rank role
agent calls	0	invocations that reached a model
agent latency p50 / p95	0.00 / 0.00 s	per invocation
messages	8	0 eager, 8 rendezvous
tokens sent	344,928	344,928 deferred under rendezvous
tokens in / out	0 / 0	prompt and completion
cost	$0.0000	0.0000 $/kTok out

3 What the analysis notices

Nothing anomalous was detected mechanically.

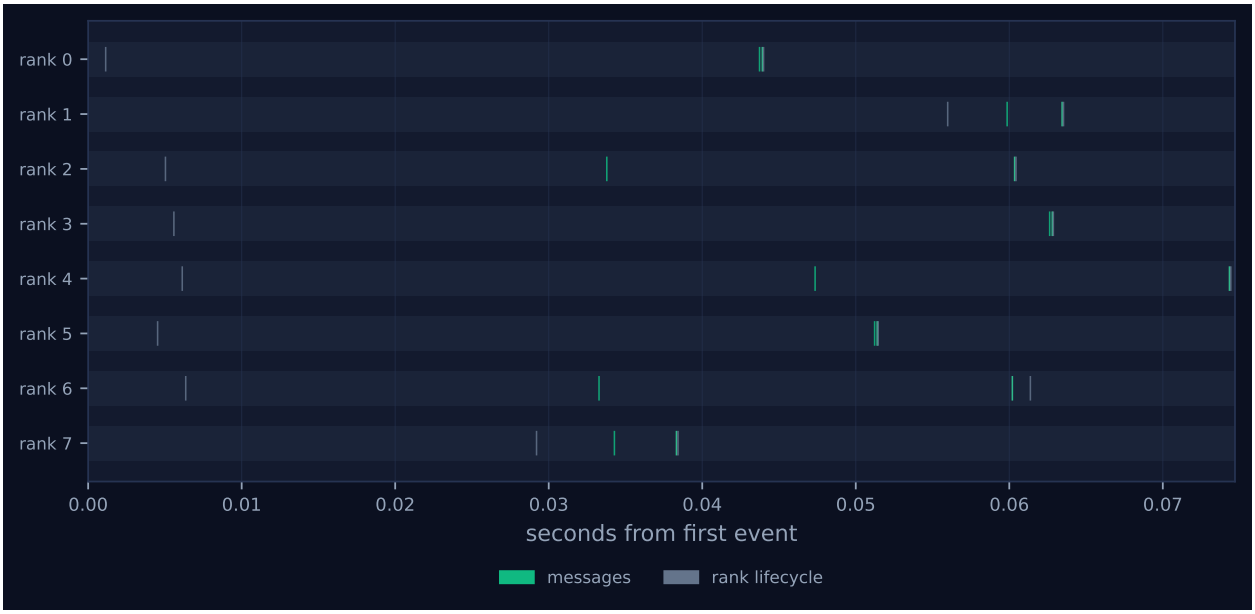

Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

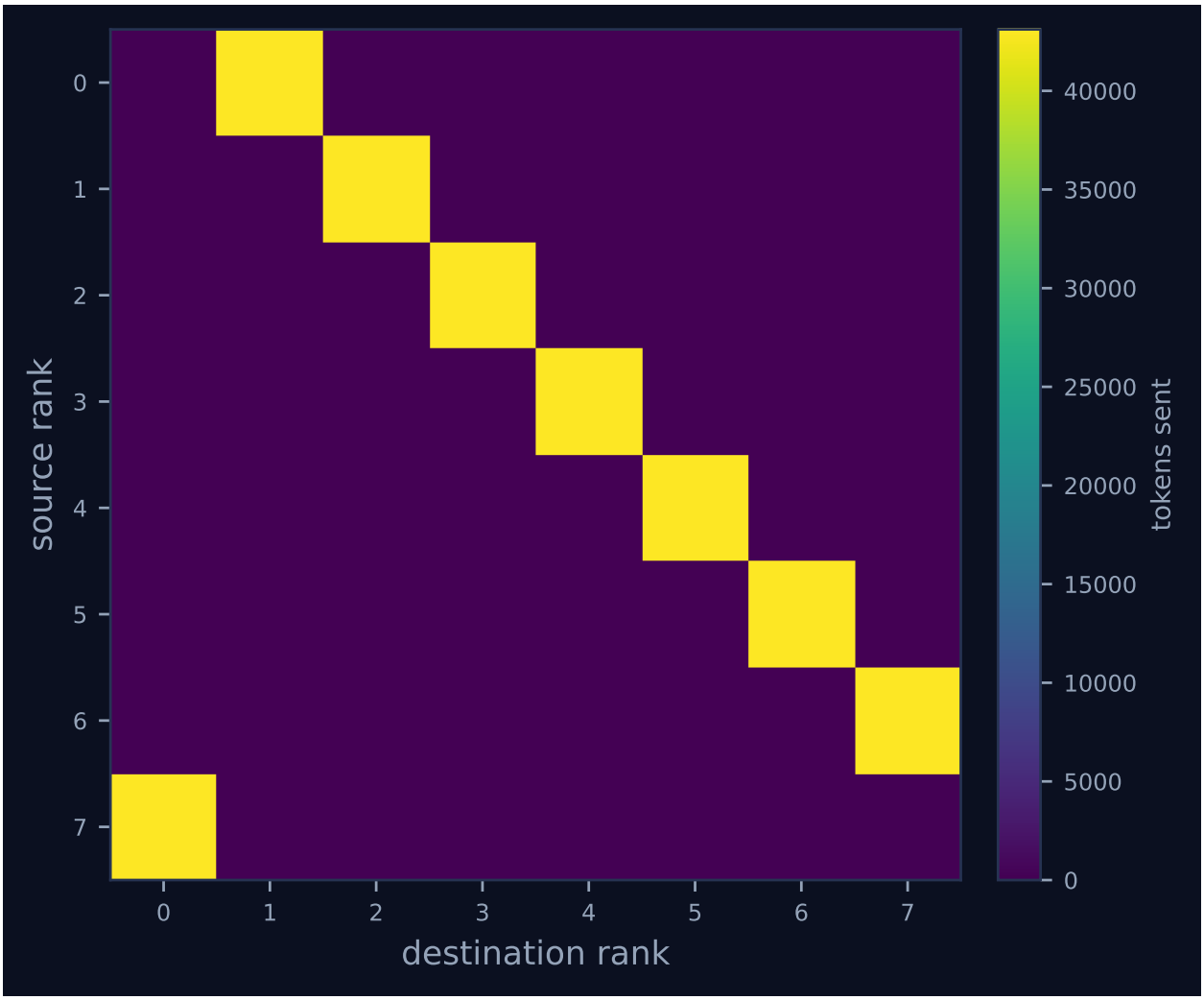

Figure 2: Communication matrix: tokens sent from each rank to each rank. The algorithm's shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	0.00	0.0	0	0	1	1	43,116	0.0	finalized
1	0.00	0.0	0	0	1	1	43,116	0.0	finalized
2	0.00	0.0	0	0	1	1	43,116	0.0	finalized
3	0.00	0.0	0	0	1	1	43,116	0.0	finalized
4	0.00	0.0	0	0	1	1	43,116	0.0	finalized
5	0.00	0.0	0	0	1	1	43,116	0.0	finalized
6	0.00	0.0	0	0	1	1	43,116	0.0	finalized
7	0.00	0.0	0	0	1	1	43,116	0.0	finalized

Per-rank behaviour. Occupancy is busy time over the rank's own lifetime, so a rank that joined late is not penalised for the time before it existed.

5 Collectives, against the cost model

This run invoked no collectives.

6 Reading the timeline

Figure 1 shows eight lanes with two grey lifecycle diamonds and two green message ticks each and no bars, which is a property of an agent-free benchmark: the executor is `none` on all eight ranks, there were zero agent calls, and no `task.*` event was emitted, so "idle fraction 100.0%" and "achieved parallelism 0.00×" say that no rank held a task rather than that ranks waited. This is the longest-spanning run in the sweep at 75 ms, with a 55 ms rank start-up window and receive latencies of 37 ms median and 53 ms maximum, an order of magnitude above the 2.5 ms median at 128 words. None of that is the payload arriving: a rendezvous receive transfers nothing, and it is the ring's own ordering that inflates the figure. Every rank sends before it receives, and `Communicator.send` calls `fabric.blobs.put` before it does anything else, so each rank writes 163,840 bytes to the content store before it looks in its mailbox — and that write sits inside the latency of the message already waiting there. The eager cell at this size, which sent nothing at all, spans 70 ms for the same reason: its blob was written and only then was the send refused. Wall time in this sweep tracks bytes into the content store, not transport mode.

Figure 2 shows the ring as one off-diagonal band of 43,116 tokens per edge, rank r to rank $(r{+}1)$ mod 8. The eager cell at this size has no such figure, because there is no traffic to plot. The row that carries the result is in the rank health table: 0% context occupancy on all eight ranks, with every `msg.recv` recording `admitted: false` and `admitted_tokens: 0` against `tokens: 43116`, and every `rank.finalize` recording `admissions: 0`, `n_items: 0`, `high_water: 0`, `used: 0`. The stats row reads `TOKENS DEFERRED 344.9k`. Set against the eager column, where occupancy climbs 1%, 2%, 8% across 128, 512 and 2,048 words and then stops because the runs stop, the rendezvous column is a flat zero from 128 words to 32,768 — a factor of 256 in payload with no change in the receiver's context at all.

7 Interpretation

By construction: the ring, eight ranks, one send and one receive each, and the mode. Note that the mode here is what AgentMPI would have chosen unprompted. Every `rank.init` records `eager_limit: 1000000000`, disabling `_choose_mode`'s threshold, but the shipped default is `DEFAULT_EAGER_LIMIT` = 2048 tokens and this payload is 43,116, so the library's own policy is rendezvous by a factor of twenty-one. The eager cell at this size is the harness overriding that policy, and the override does not merely under-perform — it cannot work.

The reason it cannot work is the point of this cell, and it is a stronger statement than the one available one size down. At 8,192 words the eager payload is 10,779 tokens: over the 4,096-token unexpected-message credit but comfortably inside the 32,768-token context budget, so that pair isolates a flow-control ceiling and shows AgentMPI refusing a message the receiver could actually have held. Here the payload is 43,116 tokens, which exceeds the receiver's entire context window. Even with unlimited eager credit the exchange is impossible: the message would have arrived, `_deliver` would have called `rt.admit`, and `ContextAccount.admit` would have found `would_fit` false against a budget of 32,768 with `strict_context` true, failing with `ERR_TRUNCATE` instead of `ERR_CONTEXT_OVERFLOW`. There is no configuration of the eager path that delivers this payload, because eager's contract is that the payload enters the receiver's context and the payload does not fit. And yet the artefact moved: all eight envelopes were delivered, the blob `04af1d57ed51` sits in the fabric at its full 163,840 bytes, byte-identical to the one the failed eager run wrote, and any receiver could have called `fetch` with a `View` to take a projection of it that did fit. This is what `comm.py` means when it says MPI hides the eager/rendezvous choice because both modes deliver the same bytes while AgentMPI exposes it because they do not have the same effect on the receiving agent's context. The bytes were the same in both runs. The effect was total.

The deferred figure needs a correction, and this cell shows it at its largest. `metrics.json` reports 344,928, which is $8 \times 43{,}116$: one rendezvous send per rank, exactly equal to `tokens sent`, as it must be when every message is rendezvous, and 4.00× the 8,192-word cell for 4× the payload. The results file records 689,856, and the per-rank events say why — `tokens_deferred: 86232` against `tokens_sent: 43116`, a rank deferring twice what it sent, which is impossible for a quantity documented as a subset of transmitted traffic. Until commit `0fea51d2`, `RankCost.tokens_deferred` accumulated on rendezvous sends and on every non-admitted receive alike, so each rank charged its own send and its predecessor's arrival to one counter. The fix separated `tokens_deferred` (send-side, rendezvous only) from `tokens_unadmitted` (receive-side, transport-agnostic), under which this run reports 344,928 of each. `cost.summarise` reconstructs from the log and was always correct, so the viewer and `metrics.json` already show the right number; only the archived results carry the doubled one. The fix's summary reasons that the paper's transport table is unaffected because the microbench admits its receives — correct for the eager rows, which read zero, and incorrect for the rendezvous rows, because `bench_transport` passes no `admit` argument and `_deliver` defaults it to `False` for rendezvous. All five rendezvous entries in that table are exactly 2× the send-side deferral.

8 Threats to this reading

The headline — 43,116 tokens moved into a 32,768-token receiver at zero cost — is true and incomplete, and the incompleteness is the main threat. Nothing in this sweep calls `fetch`. The receivers finished holding a digest, a token count, a kind and a one-line synopsis of a document none of them read, so what is demonstrated is that a large artefact can be *addressed* without

consuming context, not that it can be *used* without consuming context. A rank that needed the whole thing would find it does not fit and would have to project it through a `View` or release something first, and no cell in this sweep exercises either. The claim that eager could not have worked at any credit setting is a code-path argument from `rank.py`, not a measurement — it was not run — although the arithmetic behind it, 43,116 against 32,768, is not in question. Token counts are estimates: provenance records `tokenizer_exact: false`, and 43,116 is $\lceil 163840/3.8 \rceil$ from a character heuristic applied to one repeated word. A real BPE tokeniser would price this text near 32,768 tokens, which would put it essentially *at* the context budget rather than 32% above it, weakening the second-ceiling argument to a marginal case while leaving the first ceiling — an order of magnitude above the credit either way — untouched. Both budgets are benchmark parameters rather than library defaults (`unexpected_limit` 4,096 against a shipped 8× eager limit; `context_budget` 32,768 against 128,000), so the ratio between payload and window is one the benchmark chose. And the executor is `none` with zero agent calls, so the 0% occupancy is an entry in `ContextAccount` rather than an observation about a model, while the 75 ms wall time is one sample of eight SQLite-backed rank start-ups and eight 160 KiB blob writes.